SECURE CYBER DASHBOARD

PRD · SRS · FS — Combined Specification

Document Control

Document Type	Combined PRD / SRS / FS
Version	1.0.0
Date	May 20, 2026
Author	Eden Alon
Status	Draft — Portfolio Release
Confidentiality	Public — GitHub Portfolio

Abstract

This document is the authoritative specification for the **Secure Cyber Dashboard** — a full-stack web application providing security teams with a centralised, role-aware interface for monitoring alerts, managing user accounts, and visualising threat data. The document deliberately combines three specification formats: a **Product Requirements Document (PRD)** outlining business intent and success metrics; a **Software Requirements Specification (SRS)** containing formal, testable shall-statements and non-functional requirements; and a **Functional Specification (FS)** detailing concrete screen states, validation rules, and per-endpoint API contracts. Items are clearly labelled **Implemented**, **Planned**, or **Assumption (verify)** to maintain honesty about the current state of the codebase.

Table of Contents

PART A — PRD — Product Requirements Document

Why we are building this, who it is for, and what success looks like.

A.1 Executive Summary

Problem. Security Operations teams lack a unified, authenticated web interface to triage alerts, manage analyst accounts, and review key threat statistics. Existing tools are either enterprise-licensed, overly complex, or not tailored for small-to-medium SOC teams.

Solution. Secure Cyber Dashboard is an open-source, self-hostable full-stack web application. A React/TypeScript SPA consumes a hardened ASP.NET Core 9 REST API backed by PostgreSQL. The system provides JWT-authenticated login, a live metrics dashboard, alert management, and a user administration panel — all wrapped in a clean, dark-themed UI optimised for operational clarity.

Outcomes sought. Reduced mean-time-to-acknowledge (MTTA) for alerts; a reusable, demonstrably secure codebase that serves as a portfolio artefact; and a foundation that can be extended with real data sources (SIEM integrations, threat feeds).

A.2 Target Users & Personas

Analyst — Alex — Security Operations

- Goals: Triage incoming alerts, view dashboard KPIs, update alert status.
- Pain points: Cluttered tools, excessive false positives, no mobile-friendly view.

SOC Manager — Maya — Team Lead / Admin

- Goals: Manage analyst accounts, review aggregated metrics, export reports.
- Pain points: Lack of role separation, no audit trail for account changes.

Developer / Evaluator — Dan — Portfolio Reviewer / Hiring Manager

- Goals: Assess code quality, architecture decisions, and security posture.
- Pain points: Undocumented APIs, no local-run guide, ambiguous implemented vs planned features.

A.3 Goals & Non-Goals

Goals (in scope)

- Secure JWT-based authentication with BCrypt password hashing.
- Role-based access control: Analyst vs Admin roles.
- Real-time-style alert listing with CRUD operations.
- Dashboard page with aggregate statistics (total alerts, severity breakdown, user count).
- User management panel (Admin only): list, create, deactivate users.
- Fully documented REST API via Swagger/OpenAPI.
- Containerised backend (Docker) + static SPA deployment (e.g. Vercel).

Non-Goals (explicitly out of scope)

- Live SIEM/EDR integration (planned Phase 3).
- Mobile native application.
- End-to-end encryption of alert payload at rest (database-level encryption is a future concern).
- Multi-tenancy / organisation isolation.

• SOC2 / ISO 27001 certification.

A.4 User Journeys

Journey 1 — Analyst morning triage

• Alex opens the dashboard URL and is redirected to /login.
• Alex enters credentials; the SPA sends POST /api/auth/login and receives a JWT.
• On success, Alex is routed to /dashboard showing today's alert counts and severity chart.
• Alex navigates to /alerts, filters by 'Critical' severity, and opens alert detail.
• Alex sets status to 'Acknowledged', adds a note, and saves — PATCH /api/alerts/{id}.
• Alex logs out; JWT is removed from storage and the session ends.

Journey 2 — Admin account provisioning

• Maya logs in (Admin role) and navigates to /admin/users.
• Maya clicks 'Add User', fills the form (name, email, role, temp password).
• SPA sends POST /api/users; backend validates, hashes password, persists record.
• New analyst appears in the user list with 'Active' status.
• Maya deactivates a former employee — PATCH /api/users/{id} sets isActive=false.

Journey 3 — Developer evaluation

• Dan clones the repo and follows the Local Setup appendix.
• Dan runs docker compose up and npm run dev; reaches the app at localhost:5173.
• Dan opens /swagger and exercises all endpoints, noting clear auth requirements.
• Dan reviews the codebase: Clean Architecture layers, EF Core migrations, Axios services.

A.5 Scope — MVP vs Later Phases

Phase	Scope	Status
MVP (Phase 1)	Auth (login/register/logout), JWT issuance & refresh, Dashboard metrics, Alert CRUD, Role-based routes	Implemented
Phase 2	Alert filtering/search/pagination, audit log, email notifications on critical alerts, CSV export	Planned
Phase 3	SIEM webhook ingestion, threat intelligence feed display, OAuth2 SSO, advanced charting	Planned

A.6 Success Metrics (Indicative — not yet instrumented)

Metric	Target	Measurement
Login success rate	> 99%	Auth error logs
Mean time to acknowledge alert (MTTA)	< 5 min (demo env)	Alert timestamps
Lighthouse Performance score	> 85	Chrome DevTools
API p95 response time	< 300 ms (local)	Swagger manual test
Zero critical CVEs in dependencies	0 findings	npm audit / dotnet list

A.7 Roadmap

Quarter	Phase	Deliverables	Status
Q1	Phase 1 — MVP	Auth, Dashboard, Alert CRUD, Docker, Swagger	Complete
Q2	Phase 2 — Operations	Pagination, audit log, notifications, export	Planned
Q3	Phase 3 — Intelligence	SIEM ingestion, threat feeds, SSO	Planned

PART B — SRS — Software Requirements Specification

Formal, testable requirements. IDs are stable and referenced in Part C.

B.1 System Context & Stakeholders

The system consists of two deployment units: (1) a **React SPA** served from a CDN/static host, communicating with (2) an **ASP.NET Core Web API** running in Docker, backed by a **PostgreSQL 15** database. No third-party analytics or external data sources are used in Phase 1.

Figure B-1 — System Architecture Context Diagram

Stakeholder	Role
Security Analyst	Primary end-user; reads and updates alerts.
SOC Manager / Admin	Manages users; views aggregate metrics.
Developer / Ops	Deploys and maintains the system.
Portfolio Reviewer	Evaluates architecture, security, and code quality.

B.2 Glossary

Term	Definition
JWT	JSON Web Token — a signed, self-contained bearer token used for stateless authentication.
BCrypt	Adaptive password hashing algorithm; work factor configurable (default 12).
Alert	A security event record with severity, status, title, description, and timestamp.
Role	An RBAC classification: Analyst (read/update) or Admin (full CRUD + user management).
SPA	Single-Page Application — the React frontend.
EF Core	Entity Framework Core — the ORM used for Code-First database migrations.
DTO	Data Transfer Object — shape of JSON exchanged between SPA and API.
MTTA	Mean Time To Acknowledge — key alert-response latency metric.

B.3 Overall Description

Product perspective. Secure Cyber Dashboard is a self-contained internal tool, not a SaaS product. It is designed to be forked, adapted, and deployed on private infrastructure.

Constraints.

- Must run on commodity hardware (4 GB RAM, 2 vCPU minimum for Docker + Postgres).
- Backend targets .NET 9 LTS. Frontend targets ES2020+ browsers (Chrome 110+, Firefox 115+, Edge 110+).
- All sensitive secrets (JWT secret, DB connection string) must be passed via environment variables — never hard-coded.
- The API must remain stateless: no server-side sessions.

Dependencies.

- ReportLab / Vite 5+ (build tooling).
- Docker Engine 24+ for containerised API.
- PostgreSQL 15+.
- Node.js 20+ (frontend dev / build).
- .NET SDK 9 (backend dev / build).

B.4 Functional Requirements

ID	Requirement (The system shall…)	Status
FR-001	Provide a POST /api/auth/register endpoint that accepts email, password, and role; validates input; hashes the password with BCrypt; and persists a new User record.	**Implemented**
FR-002	Provide a POST /api/auth/login endpoint that validates credentials and returns a signed JWT with userId, email, and role claims.	**Implemented**
FR-003	Invalidate the client JWT on logout (client-side removal; server-side blocklist is Phase 2).	**Implemented**
FR-004	Protect all non-auth endpoints with JWT Bearer authentication middleware.	**Implemented**
FR-005	Enforce role-based access: Admin endpoints must reject Analyst tokens with 403 Forbidden.	**Implemented**
FR-006	Expose GET /api/alerts returning a paginated list of alerts for the authenticated user's scope.	**Implemented**
FR-007	Expose POST /api/alerts allowing Admins and Analysts to create a new alert with title, description, severity, and status.	**Implemented**
FR-008	Expose PATCH /api/alerts/{id} allowing status and note updates; validate that the caller owns or is Admin.	**Implemented**
FR-009	Expose DELETE /api/alerts/{id} restricted to Admin role.	**Implemented**
FR-010	Expose GET /api/dashboard/stats returning total alert count, counts by severity, and active user count.	**Implemented**
FR-011	Expose GET /api/users (Admin only) returning a paginated list of user accounts.	**Implemented**
FR-012	Expose POST /api/users (Admin only) to create a new user account.	**Implemented**
FR-013	Expose PATCH /api/users/{id} (Admin only) to update role or deactivate a user (isActive flag).	**Implemented**
FR-014	Serve a Swagger/OpenAPI UI at /swagger in non-production environments.	**Implemented**
FR-015	The SPA shall redirect unauthenticated users from protected routes to /login.	**Implemented**
FR-016	The SPA shall display a loading skeleton while API requests are in-flight.	**Implemented**
FR-017	The SPA shall display user-friendly error messages on API 4xx/5xx responses.	**Implemented**
FR-018	Provide alert filtering by severity and status via query parameters (Phase 2).	**Planned**

FR-019	Provide full-text search across alert titles and descriptions (Phase 2).	**Planned**
FR-020	Generate and download alert reports as CSV (Phase 2).	**Planned**

B.5 Non-Functional Requirements

ID	Requirement (The system shall…)	Status
NFR-001	All passwords shall be hashed using BCrypt with a minimum work factor of 12 before persistence.	**Implemented**
NFR-002	JWTs shall be signed with HS256 and expire after 60 minutes; secret ≥ 256 bits via env var.	**Implemented**
NFR-003	The API shall enforce HTTPS in production; HTTP requests shall be redirected.	**Assumption (verify)**
NFR-004	CORS policy shall whitelist only the known SPA origin(s); wildcard (*) is forbidden in production.	**Implemented**
NFR-005	All database queries shall use parameterised queries via EF Core to prevent SQL injection.	**Implemented**
NFR-006	API p95 response time shall be < 500 ms under a load of 50 concurrent users on reference hardware.	**Assumption (verify)**
NFR-007	The SPA Lighthouse Performance score shall target ≥ 85 on desktop.	**Assumption (verify)**
NFR-008	The system shall achieve ≥ 99.5% uptime during business hours in a single-node deployment.	**Assumption (verify)**
NFR-009	EF Core Code-First migrations shall be the sole mechanism for schema changes; no manual DDL.	**Implemented**
NFR-010	All API error responses shall use RFC 9457 Problem Details JSON format.	**Implemented**
NFR-011	The frontend shall be accessible at WCAG 2.1 AA level for core user flows.	**Planned**
NFR-012	Docker image shall be reproducible from source with a single docker compose up command.	**Implemented**

B.6 External Interfaces

Browser ↔ API. JSON over HTTPS. All requests include Authorization: Bearer <token> except /api/auth/*. Content-Type: application/json. CORS preflight handled by ASP.NET Core middleware.

API ↔ PostgreSQL. EF Core connection pool; connection string via ASPNETCORE env var. No direct SQL; all queries via LINQ/EF Core.

Developer ↔ Swagger. OpenAPI 3.0 spec auto-generated by Swashbuckle; accessible at /swagger/index.html in Development profile.

B.7 Data Dictionary (High-Level)

Entity	Fields	Sensitive Field Policy
User	Id (UUID), Email (unique, indexed), PasswordHash, Role (Analyst|Admin), IsActive, CreatedAt	PasswordHash — never exposed in API responses.
Alert	Id (UUID), Title, Description, Severity (Low|Medium|High|Critical), Status (Open|Acknowledged|Resolved|Closed), CreatedById (FK→User), CreatedAt, UpdatedAt	No PII by design in Phase 1.

RefreshToken	Id, Token (hashed), UserId (FK), ExpiresAt, IsRevoked	Phase 2 — not yet implemented.

B.8 Security & Privacy Requirements

Threat assumptions. The primary threats modelled are: credential brute-force, JWT forgery, SQL injection, XSS via reflected data, and CSRF. The system is not designed to defend against nation-state adversaries or insider threats at the database level.

Credential brute-force: Rate limiting on /api/auth/* (Phase 2); BCrypt slow hashing degrades attack speed.

JWT forgery: HS256 with a ≥ 256-bit secret from environment; tokens expire in 60 min.

SQL injection: All queries via EF Core parameterised LINQ — no raw SQL.

XSS: React's default JSX escaping; no dangerouslySetInnerHTML usage; Content-Security-Policy header (Phase 2).

CSRF: JWT in memory/httpOnly cookie (not localStorage) eliminates the classic CSRF vector; SameSite cookie policy (verify).

Sensitive data exposure: PasswordHash excluded from all User DTOs via explicit mapping; no logging of tokens.

B.9 Traceability Table

FR/NFR IDs	Feature Area	FS Reference
FR-001, FR-002, FR-003	Auth flow	C.1, C.6 POST /api/auth/*
FR-004, FR-005	JWT middleware, RBAC	C.7 Auth Session Model
FR-006 – FR-009	Alert CRUD	C.3, C.6 /api/alerts
FR-010	Dashboard stats	C.2, C.6 /api/dashboard/stats
FR-011 – FR-013	User management	C.6 /api/users
FR-015 – FR-017	SPA routing & error states	C.4 UI Specification
NFR-001 – NFR-005	Security controls	B.8, C.5 Validation Rules

PART C — FS — Functional Specification

Concrete behaviour: flows, screen states, validation, API contracts, edge cases.

C.1 Authentication Flow

Figure C-1 — Login Sequence Diagram

Actor	SPA	API	DB
User enters credentials	■ POST /api/auth/login {email, password}		
		■ SELECT User WHERE email	
			■ User row
		BCrypt.Verify() Issue JWT (HS256)	
	■ {token, expiresIn}		
User sees Dashboard	Store token, route to /dashboard		

Preconditions: User account exists and isActive=true.

Steps:

- User navigates to /login (or is redirected from a protected route).
- User enters email and password; client validates non-empty fields before submission.
- SPA calls POST /api/auth/login; shows spinner on button.
- API looks up User by email; returns 401 if not found.
- API calls BCrypt.Verify(inputPassword, storedHash); returns 401 on mismatch.
- API issues a signed JWT containing { sub, email, role, exp }.
- SPA stores JWT in memory (and optionally httpOnly cookie — assumption: verify implementation).
- SPA sets Axios default header Authorization: Bearer .
- SPA routes to /dashboard.

Postconditions: JWT is stored; user is on /dashboard; all subsequent API calls include the token.

Failure paths:

- Invalid credentials → 401 → SPA shows inline error 'Invalid email or password.'
- Account inactive → 403 → SPA shows 'Your account is disabled. Contact your administrator.'
- Network error → SPA shows 'Unable to reach the server. Please try again.'

C.2 Dashboard Flow

- On mount, **DashboardPage** dispatches GET /api/dashboard/stats.
- While awaiting: stat cards show skeleton loaders (pulsing grey boxes).
- On success: cards render Total Alerts, Critical Count, High Count, Active Users.
- Severity chart (bar or donut) renders from the severityCounts map.
- On API error: an inline alert banner reads 'Failed to load dashboard data. Refresh to retry.'
- Auto-refresh interval: none in Phase 1; manual refresh button present (Assumption: verify).

C.3 Alert Management Flow

List view (/alerts). Table columns: ID (truncated UUID), Title, Severity badge, Status badge, Created At, Actions. Default sort: CreatedAt DESC.

Create alert (modal/drawer).

- User clicks 'New Alert'; a modal opens with fields: Title*, Description*, Severity* (dropdown), Status* (defaults to Open).
- Client validates: title 3–200 chars, description 1–2000 chars, severity in enum.
- On submit: POST /api/alerts. 201 → alert appended to list; modal closes.
- On validation error from server: field-level error messages displayed.

Update alert status.

- User clicks a row → detail panel opens showing all fields.
- Status dropdown allows transitions: Open → Acknowledged → Resolved → Closed.
- User optionally adds a note; clicks 'Save' → PATCH /api/alerts/{id}.
- 204 response → optimistic UI update; toast notification 'Alert updated.'

Delete alert (Admin only).

- Admin clicks trash icon → confirmation dialog → DELETE /api/alerts/{id} → row removed from list.

C.4 UI Specification

Route	Guard	Component	Key Content
/login	Public	LoginPage	Email input, password input, submit button, error banner. No navigation bar.
/dashboard	Auth	DashboardPage	Stat cards (4x), severity chart, recent alerts table (top 5). Sidebar nav.
/alerts	Auth	AlertsPage	Paginated table, filters (Phase 2), 'New Alert' button, row actions.
/admin/users	Admin	UsersPage	User table, 'Add User' button, deactivate toggle, role badge.
/profile	Auth	ProfilePage	Display name, email (read-only), change password form (Assumption).
*	—	NotFoundPage	404 illustration, 'Go to Dashboard' CTA.

Design tokens (assumptions — verify against codebase).

- Color scheme: dark navy background (#0D1B2A), steel card surfaces, accent blue (#2A7FD4).
- Typography: Inter or system-ui; headings 600 weight; body 400 weight; code Fira Code.
- Severity badges: Critical=red, High=orange, Medium=yellow, Low=green.
- Status badges: Open=blue, Acknowledged=purple, Resolved=green, Closed=grey.
- Responsive breakpoints: mobile-first; sidebar collapses to hamburger < 768px.

C.5 Validation Rules

Field	Layer	Rule	Error Response
Email	Client + Server	Valid RFC 5322 format; max 255 chars; unique in DB.	400 with field error
Password (register)	Client + Server	Min 8 chars; at least 1 uppercase, 1 digit, 1 special char.	400 with field error
Alert Title	Client + Server	3–200 chars; not blank.	400 with field error
Alert Description	Client + Server	1–2000 chars.	400 with field error

Severity	Client + Server	Enum: Low | Medium | High | Critical.	400 with field error
Status	Server	Enum: Open | Acknowledged | Resolved | Closed. Transitions validated server-side.	422 Unprocessable
User Role	Server (Admin only)	Enum: Analyst | Admin.	400 with field error
UUID path params	Server	Must be valid UUID v4; reject non-parseable values.	400 Bad Request

C.6 API Specification

[■ Assumption: All endpoints return application/json. Base URL: https:///api]

Authentication Endpoints

POST `/api/auth/register`	Auth: None
Request	{ email: string, password: string, role: 'Analyst'|'Admin' }
Response	{ userId: uuid, email: string, role: string, createdAt: ISO8601 }
Status codes	201 Created | 400 Validation error | 409 Email already registered
Notes	Password is never returned. BCrypt hash stored only.

POST `/api/auth/login`	Auth: None
Request	{ email: string, password: string }
Response	{ token: string (JWT), expiresIn: number (seconds), role: string }
Status codes	200 OK | 401 Invalid credentials | 403 Account inactive

Alert Endpoints

GET `/api/alerts`	Auth: Bearer JWT (Analyst|Admin)
Request	Query: ?page=1&pageSize;=20&severity;=Critical&status;=Open (Phase 2 filters)
Response	{ data: Alert[], total: number, page: number, pageSize: number }
Status codes	200 OK | 401 Unauthorized | 403 Forbidden

POST `/api/alerts`	Auth: Bearer JWT (Analyst|Admin)
Request	{ title: string, description: string, severity: string, status: string }
Response	Alert object (full)
Status codes	201 Created | 400 Validation | 401 Unauthorized
Notes	createdById set from JWT sub claim on server side.

PATCH `/api/alerts/{id}`	Auth: Bearer JWT (Analyst|Admin)
Request	{ status?: string, note?: string }

Response	204 No Content
Status codes	204 | 400 | 401 | 403 | 404 Not Found
Notes	Analyst can update own alerts only. Admin can update any.

DELETE `/api/alerts/{id}`	Auth: Bearer JWT (Admin only)
Request	—
Response	204 No Content
Status codes	204 | 401 | 403 | 404

Dashboard Endpoint

GET `/api/dashboard/stats`	Auth: Bearer JWT (Analyst|Admin)
Request	—
Response	{ totalAlerts: number, bySeverity: { Low, Medium, High, Critical }, activeUsers: number }
Status codes	200 OK | 401 Unauthorized

User Management Endpoints (Admin)

GET `/api/users`	Auth: Bearer JWT (Admin)
Request	Query: ?page=1&pageSize;=20
Response	{ data: User[], total, page, pageSize }
Status codes	200 | 401 | 403
Notes	PasswordHash field is excluded from all User DTOs.

POST `/api/users`	Auth: Bearer JWT (Admin)
Request	{ email, password, role }
Response	User object (no passwordHash)
Status codes	201 | 400 | 401 | 403 | 409 Duplicate email

PATCH `/api/users/{id}`	Auth: Bearer JWT (Admin)
Request	{ role?: string, isActive?: boolean }
Response	204 No Content
Status codes	204 | 400 | 401 | 403 | 404
Notes	Admin cannot deactivate their own account.

C.7 Auth Session Model

• **Token storage:** JWT stored in-memory (React state/context) on the SPA. If httpOnly cookie is used, confirm SameSite=Strict and Secure flags are set. localStorage is explicitly avoided to mitigate XSS token theft. (Assumption: verify implementation.)

• **Token expiry:** 60 minutes (configurable via JWT_EXPIRE_MINUTES env var). The SPA checks exp claim; if expired, redirects to /login.

• **Protected routes:** A PrivateRoute HOC / wrapper checks the token presence and role. Missing token → redirect to /login. Wrong role → redirect to /403.

• **Logout:** Client clears the token from state; Axios header is removed. No server-side token blocklist in Phase 1 (token remains technically valid until exp).

• **Refresh tokens:** Not implemented in Phase 1. Planned in Phase 2 using the RefreshToken entity.

C.8 Edge Cases & Error Handling

Scenario	Behaviour
Expired JWT	SPA detects 401 on any request → clears token → redirects to /login with ?reason=session_expired query param → login page shows banner.
Tampered JWT signature	API middleware rejects with 401 Unauthorized. Client handles same as expired.
Network offline	Axios timeout (10 s default) → SPA shows toast 'Network error — check your connection.'
Concurrent edits	No optimistic locking in Phase 1. Last write wins. Phase 2: ETag / rowversion concurrency check.
Admin self-deactivation	PATCH /api/users/{self-id} with isActive=false → API returns 400 'Cannot deactivate your own account.'
Empty alert list	AlertsPage shows an empty-state illustration with 'No alerts found. Create your first alert.' CTA.
Swagger in production	Swashbuckle middleware is guarded by env check; /swagger returns 404 in Production profile.
Rate limiting	Not implemented in Phase 1. Recommended: ASP.NET Core rate-limiting middleware on /api/auth/* in Phase 2.

APPENDIX Local Setup & Run Guide

How to run the project from source on a developer machine.

Prerequisites

- .NET SDK 9
- Node.js 20+
- Docker Engine 24+ & Docker Compose v2
- PostgreSQL 15 (or via Docker)
- Git

1. Clone the repository

```
git clone https://github.com/<your-username>/secure-cyber-dashboard.git
cd secure-cyber-dashboard
```

2. Configure environment variables

Copy the example env file and fill in your values:

```
cp .env.example .env
```

Variable	Description / Example
POSTGRES_CONNECTION_STRING	Host=localhost;Port=5432;Database=cyberdash;Username=TBD;Password=TBD
JWT_SECRET	<min 256-bit random string — generate with: openssl rand -base64 32>
JWT_EXPIRE_MINUTES	60
ASPNETCORE_ENVIRONMENT	Development
CORS_ORIGIN	http://localhost:5173

3. Run the backend (Docker)

```
cd SecureDashboard.Api
docker compose up --build
```

API available at: http://localhost:8080 | Swagger: http://localhost:8080/swagger

4. Apply EF Core migrations

```
dotnet ef database update --project SecureDashboard.Data --startup-project SecureDashboard.Api
```

5. Run the frontend

```
cd client
npm install
npm run dev
```

SPA available at: http://localhost:5173

6. Seed data (optional)

```
dotnet run --project SecureDashboard.Api -- --seed
```

Creates a default Admin account: admin@cyberdash.local / Admin@1234 (change immediately).